

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY,
BANGALORE

SOFTWARE PRODUCTION ENGINEERING
CSE 816

Mini Project: Scientific Calculator with DevOps

Aieshah Nasir (MT2025017)

GitHub: <https://github.com/aie007/calculator-microservice>

Docker Hub: <https://hub.docker.com/r/aie007/calc>

March 7, 2026



International
Institute of Information
Technology Bangalore

1 Introduction

This project demonstrates a complete DevOps pipeline built around modern open-source tools. These tools, together, enable the creation, validation, packaging, and automated deployment of the Scientific Calculator Application.

1.1 DevOps

DevOps is the integration and automation of software development and information technology operations. It is characterized by key principles: shared ownership, workflow automation, and rapid feedback. Continuous integration refers to the frequent merging of several small changes into a main branch. Continuous delivery refers to producing software in short cycles with high speed and frequency so that reliable software can be released at any time, with a simple and repeatable deployment process when deciding to deploy. Continuous deployment is the automatic rollout of new software functionality. When these three processes occur in order, they are described as a "CI/CD pipeline".

1.2 Problem Statement

Create a scientific calculator program with user menu driven operations. Following are the main operations implemented using OCaml as per the requirement -

- 1 Square root function - $\sqrt{x}$
- 2 Factorial function - $x!$
- 3 Natural logarithm (base e) - $\ln x$
- 4 Power function - x^b

Alongside, addition, subtraction, multiplication and division are also implemented as part of the command line application.

1.3 DevOps Tool Chain

- **OCaml** as the language for the implementation of the scientific calculator application
- **Dune** as a build tool for the OCaml application
- **OUnit2** for unit testing the OCaml application
- **GitHub** for source control management
- **Jenkins** for Continuous Integration (CI)
- **Docker** for containerization
- **Docker Hub** as the registry for Docker Image
- **Ansible** for configuration management and deployment (CD)

2 Source Control Management - GitHub

Source Code Management (SCM), also known as Version Control System (VCS), is the process of tracking modifications to a codebase, managing changes, and helping multiple contributors collaborate and resolve conflicts when merging updates. GitHub is a web-based hosting service for Git repositories that provides Source Code Management (SCM).

2.1 Repository Structure

Figure 1 describes the structure of the Git repository. The src folder contains all OCaml application and other configuration files are contained alongside.

The public repository is available at: <https://github.com/aie007/calculator-microservice>

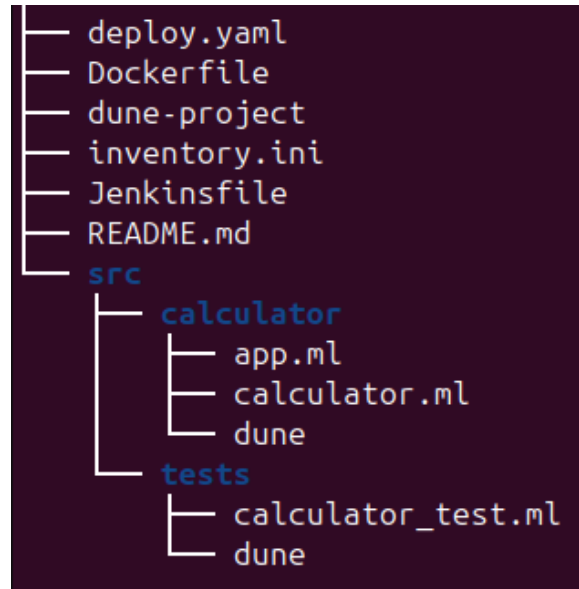


Figure 1: Repository Tree

2.2 GitHub Webhook using ngrok

The purpose of GitHub webhooks is to trigger Jenkins builds immediately on every push to main branch. We expose the local Jenkins instance (port 8080) to the internet using ngrok and configure a GitHub webhook that points to Jenkins' GitHub endpoint.

Jenkins should be running before hand on the local machine. It starts up on port 8080. Following are the steps to configure GitHub webhooks via ngrok:

- 1 Install and configure ngrok
Add your auth token (from dashboard.ngrok.com)
`ngrok config add-authtoken <YOUR_NGROK_AUTHTOKEN>`
- 2 Start a Tunnel to Jenkins
`ngrok http 8080`
- 3 Configure Jenkins Job Trigger
Open Jenkins → Your Job → Configure. In Build Triggers, check GitHub hook trigger for GITScm polling and click Save. This is done programatically using the Jenkinsfile.
- 4 Add GitHub Webhook
In the repository:
 1. GitHub → Settings → Webhooks → Add webhook
 2. Payload URL: `https://<your-ngrok>.ngrok.io/github-webhook/`
 3. Content type: `application/json`
 4. Secret (recommended): set a random string; store the same secret in Jenkins if applicable (GitHub plugin / multibranch setups that validate HMAC)
 5. Which events: Just the push event (and Pull requests if you want PR builds)
 6. Click Add webhook
- 5 Verify Delivery
GitHub sends a Ping event immediately.
In GitHub → Webhooks → Recent Deliveries, status should be 200.
In Jenkins, a new build should trigger on the next push to main.

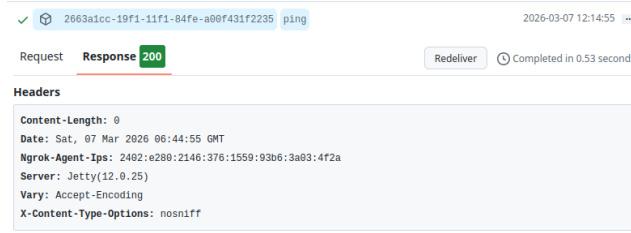


Figure 2: GitHub Webhook ping

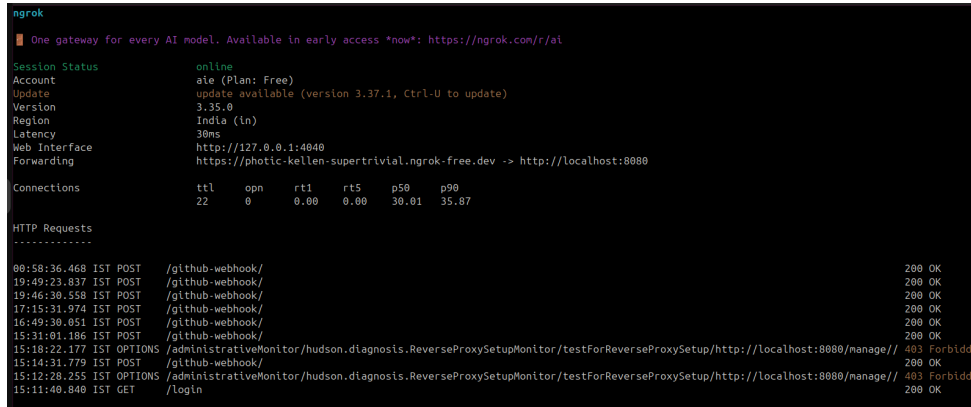


Figure 3: ngrok Configuration

3 Testing

The primary goals of testing in this project are:

- To validate all mathematical operations for accuracy.
- To catch potential errors (invalid inputs, unsupported operations).
- To enable automated test execution as part of the Jenkins pipeline.

3.1 Dune as the Build System

Dune is a high-performance, composable build system for OCaml that serves as the official, standard tool for compiling, testing, and installing projects. Opam is the package manager using which dune can be installed. dune-project file specifies the OCaml project and every folder in /src contains a dune file mentioning the modules, libraries and dependencies.

- 1 Build OCaml Application
dune build @all
- 2 Run the Calculator program
dune exec ./src/calculator/app.exe
- 3 Run unit tests
dune exec ./src/tests/calculator_test.exe

```
nasir@nasir-HP-Pavilion-Laptop-14-dv0xxx:~/Documents/courses/spe/calculator-microservice/src/tests$ dune build @all
Entering directory '/home/nasir/Documents/courses/spe/calculator-microservice'
nasir@nasir-HP-Pavilion-Laptop-14-dv0xxx:~/Documents/courses/spe/calculator-microservice/src/tests$ dune exec ./calculator_test.exe
Entering directory '/home/nasir/Documents/courses/spe/calculator-microservice'
Leaving directory '/home/nasir/Documents/courses/spe/calculator-microservice'
.....
Ran: 6 tests in: 0.11 seconds.
OK
```

Figure 4: Dune for testing

4 Dockerfile

A Dockerfile is a plain text script without an extension that contains ordered, step-by-step instructions to build a custom container image. It automates image creation, ensuring applications run identically across different environments by stacking instructions into layers

```
# Use an Ocaml image as base
FROM aie007/ubuntu-ocaml:5.4-dune-installed

# copy project source to container
COPY . /app

# set the working directory
WORKDIR /app

# set execute permissions for ocaml files
RUN chmod -R +x /app

# install dependencies
RUN opam install -y ounit2

# run calculator program
CMD ["dune", "exec", "./src/calculator/app.exe"]
```

Figure 5: Dockerfile

5 Jenkins

5.1 Jenkinsfile

A Jenkinsfile is a text file, stored in source control, that defines a Jenkins Pipeline using Groovy syntax. It enables "Pipeline as Code," allowing you to define, version, and execute automated build, test, and deploy stages. It typically lives in the root directory of a project.

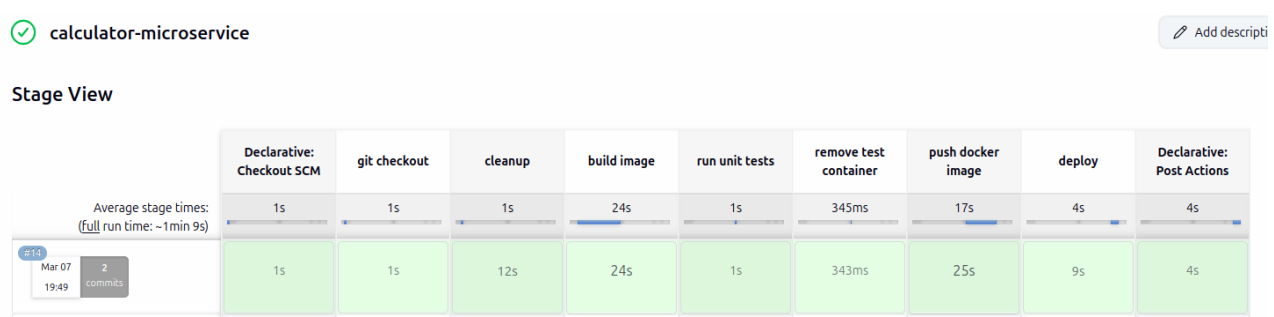


Figure 6: Jenkins Stage View

5.2 Stages

5.2.1 git checkout

Checkout the code from the repository using the provided SCM configuration

5.2.2 cleanup

Removes old containers if any exist.

5.2.3 build image

Builds the docker image using the Dockerfile in the current git repository.

5.2.4 run unit tests

The container is used to run unit tests using dune. In case of failure, post build actions will be run.

5.2.5 remove test container

The container is removed as part of cleanup.

5.2.6 push docker image

Using Docker Hub credentials, the tested image is then pushed to the registry with latest tag.

5.2.7 deploy

The Ansible playbook is run here. It is used for configuration management and deployment.

5.2.8 Declarative: Post Actions

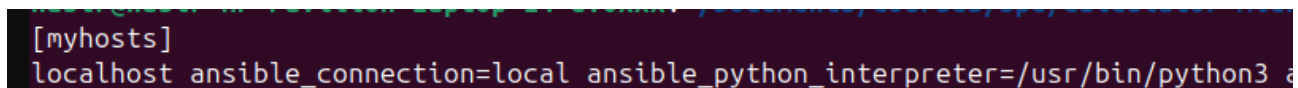
In case of success or failure, an email is sent to the configured recipients about the status of the build.

6 Ansible

Ansible is an open-source IT automation engine that automates cloud provisioning, configuration management, application deployment, and orchestration. It uses simple YAML-based playbooks, requires no agent software on managed nodes, and connects via SSH or WinRM to automate tasks across infrastructure. It is often used for infrastructure as code (IaC).

6.1 Inventory

A file defining the managed servers or network devices.

A screenshot of a terminal window showing the content of the inventory.ini file. The text is as follows:

```
[myhosts]
localhost ansible_connection=local ansible_python_interpreter=/usr/bin/python3
```

Figure 7: inventory.ini

6.2 Playbook

YAML files that define automation tasks in a human-readable format, eliminating manual, step-by-step commands. The Ansible playbook can be run locally using `ansible-playbook -i inventory.ini deploy.yaml`

```
---
- name: Deploy Calculator App with Docker
  hosts: myhosts
  remote_user: nasir
  become: false
  tasks:
    - name: Enable docker
      ansible.builtin.service:
        name: docker
        state: started

    - name: Pull calc image from docker hub
      community.docker.docker_image:
        name: 'aie007/calc:latest'
        source: pull
        register: docker_pull_result

    - name: Display docker pull result
      ansible.builtin.debug:
        var: docker_pull_result

    - name: Remove existing container if it exists
      community.docker.docker_container:
        name: calc
        state: absent

    - name: Run new container
      community.docker.docker_container:
        name: calc
        image: aie007/calc:latest
        state: started
        interactive: true
```

Figure 8: Ansible Playbook

7 Scientific Calculator Application

Upon successful deployment through Ansible automation, we can see the container up and running on our local machine.

`docker ps -a`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2bc252ae6aba	aie007/calc:latest	"dune exec ./src/cal..."	4 hours ago	Up 4 hours		calc
0d1d7f5a477b	fd6c85dbcf38	"dune exec ./src/cal..."	24 hours ago	Exited (0) 24 hours ago		happy_khayyan

Figure 9: Calc Docker Container

We can attach the 'calc' container using the following command:
`docker attach calc`

```
3
Number1: 23
Number2: 13
Result: 299.

Scientific Calculator
1. Add          2. Subtract      3. Multiply
4. Divide       5. Square Root   6. Factorial
7. Natural Log  8. Exponential  9. Exit
Enter choice: 7
Number: 89
Result: 4.48863636973

Scientific Calculator
1. Add          2. Subtract      3. Multiply
4. Divide       5. Square Root   6. Factorial
7. Natural Log  8. Exponential  9. Exit
Enter choice: 9
Exiting...
```

Figure 10: Scientific Calculator CLI

On exiting the container also exits. This behaviour can be changed by setting `restart_policy: true` in the Ansible playbook.